Lab 7: Fuzzing for Input Validation Bugs

Fuzz testing the Tennis Scoring module with Hypothesis

Name	Muhammad Affan Dumba
SRN	PES1UG24CS280
Course	Software Engineering (5th semester, CSE)
Repository	SETAPESU26/L6_L7_34_tennis_scoring (folder: lab7)
Files tested	tennis_scoring.py, test_tennis_scoring.py
Environment	Windows 11, Python 3.13.7, pytest 9.1.1, Hypothesis 6.170.15

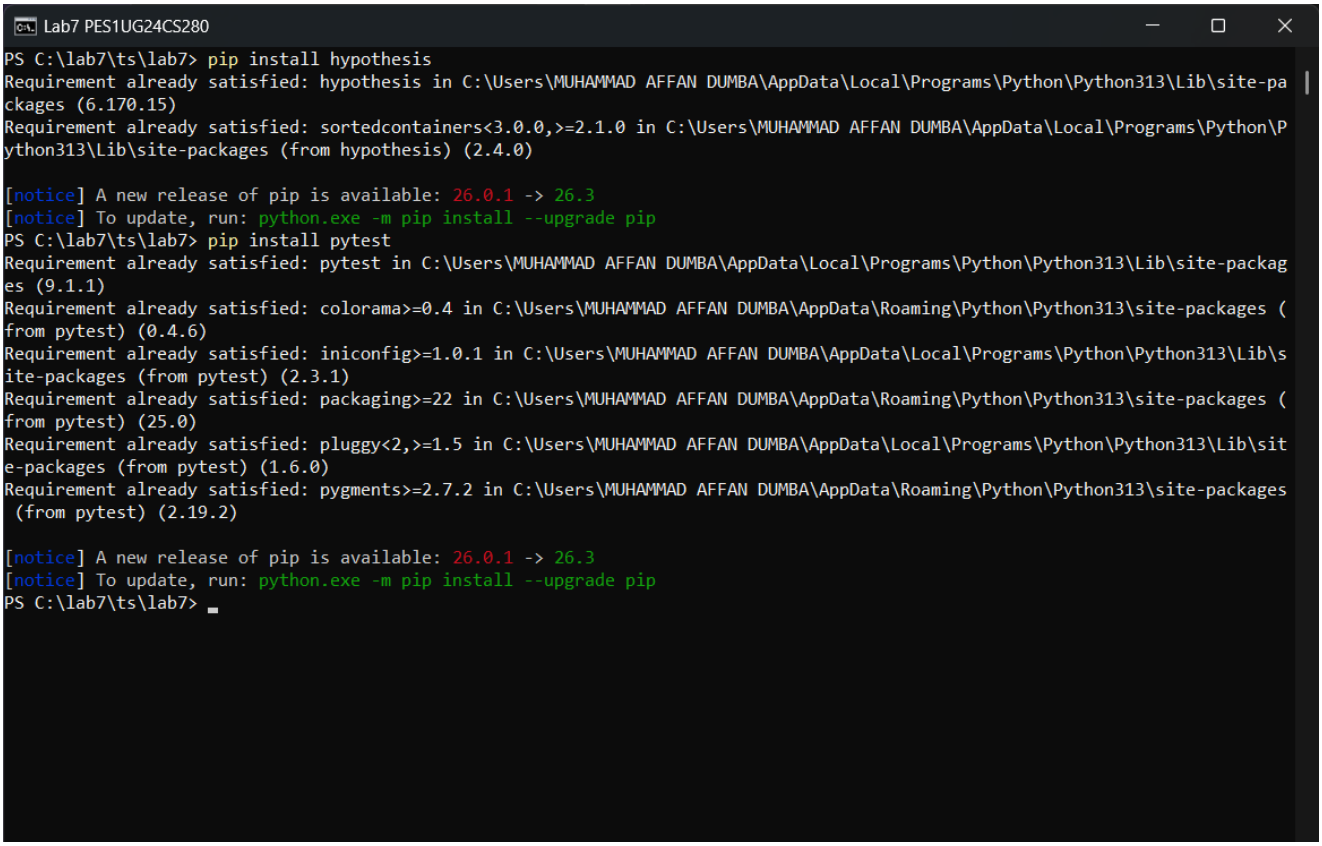
1. Aim

The aim of this lab is to use fuzz testing (property-based testing with the Hypothesis library) to find input validation bugs in the code assigned to me, and then fix the code so it handles bad input properly. My repo has a small module with three functions: `parse_score` (turns text like '3-2' into numbers), `game_lead` (lead in a game such as '40-30') and `set_spread` (max minus min of games won, from a CSV like '6,4,7').

Note: the handout talks about `processor.py` and `test_processor.py`. In my repository the same files are called `tennis_scoring.py` and `test_tennis_scoring.py`, so I used those names everywhere. Every screenshot below was taken from a real PowerShell window on my laptop while running the commands.

2. Step 1 - Installing dependencies

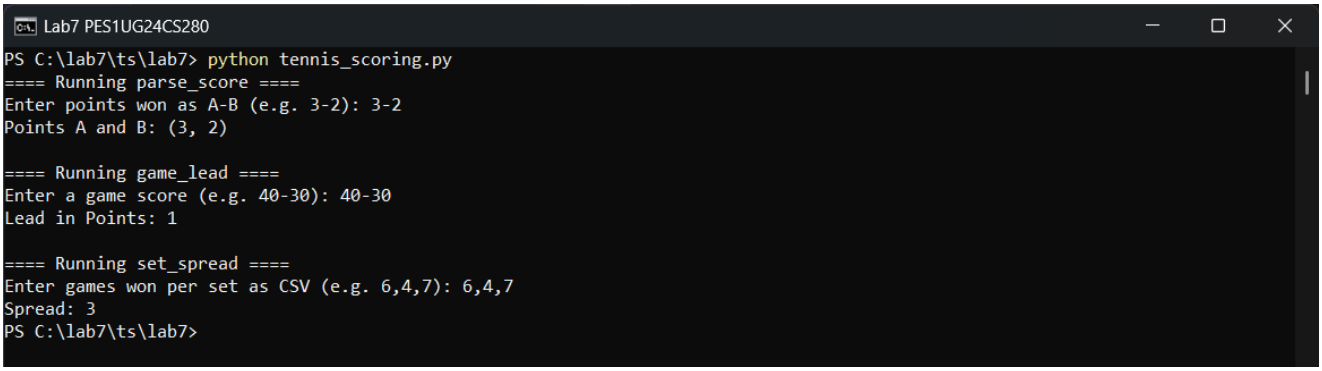
I ran `pip install hypothesis` and `pip install pytest`. Both were already installed on my laptop from earlier, so pip printed 'Requirement already satisfied' with the versions I used.



Screenshot 1: installing Hypothesis and pytest

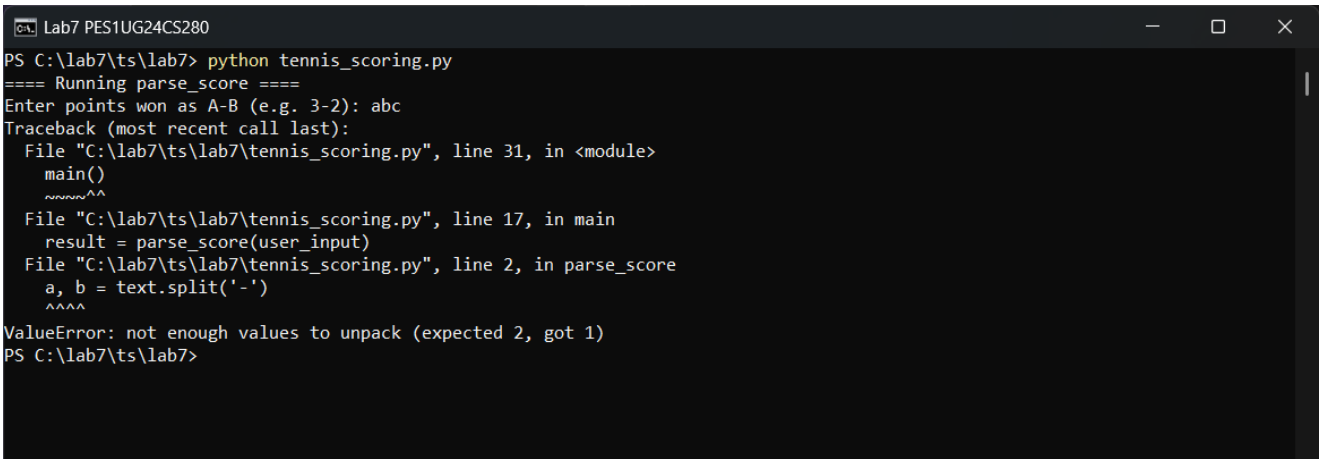
3. Step 2 - Running the original (buggy) program

I ran `python tennis_scoring.py` first with normal inputs (3-2, 40-30, 6,4,7). Everything works for well-formed input:



Screenshot 2: original program with valid input

Then I tried a wrong input ('abc') for the first prompt. The program crashes with an ugly traceback instead of telling the user what went wrong:



Screenshot 3: original program crashing on input 'abc' (ValueError)

4. Step 3 - Completing test_tennis_scoring.py

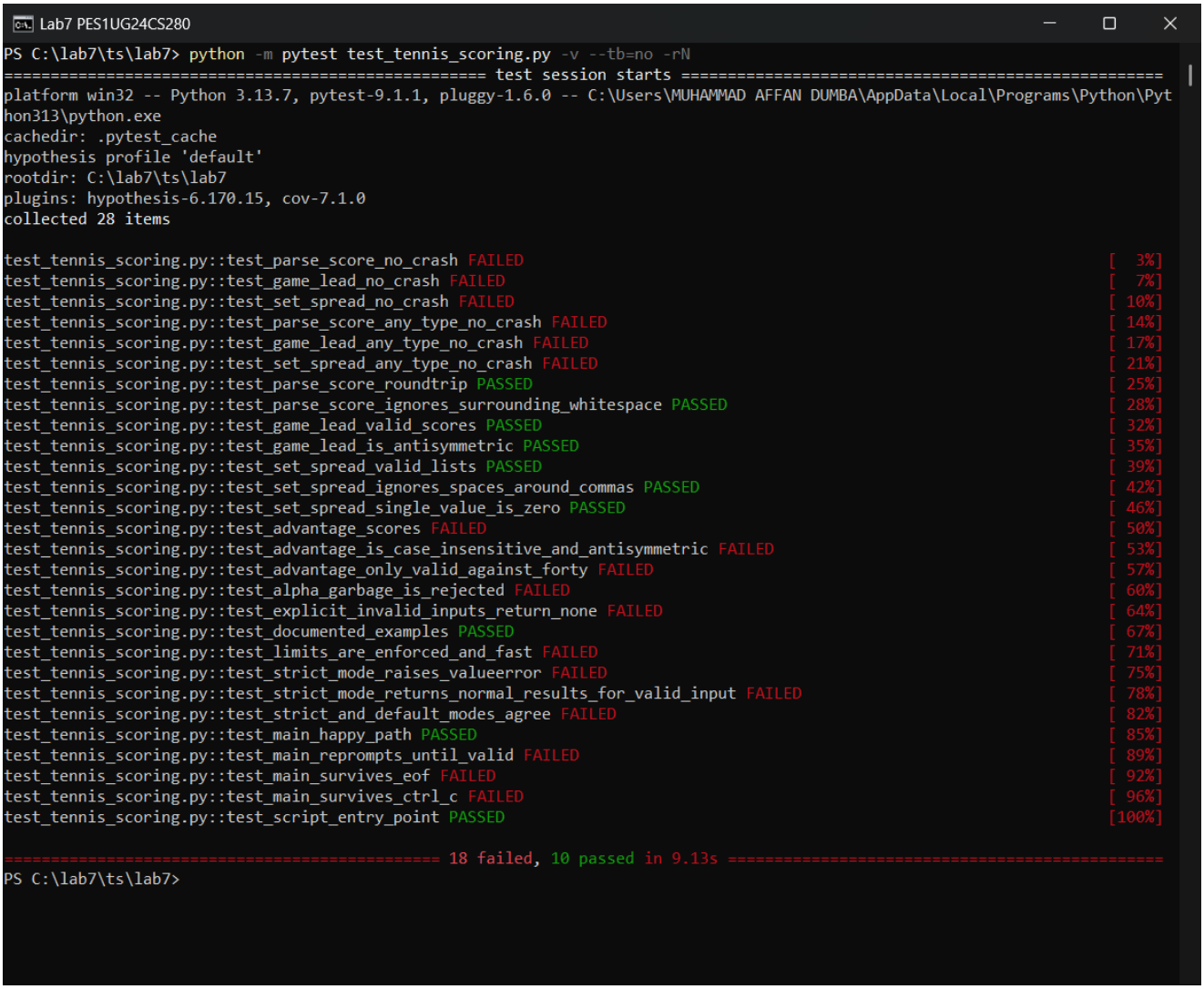
The starter file only had three empty test functions. I completed them using the pattern from the handout: call the function inside a `try/except` and fail the test if any exception comes out, with `@given(st.text() | st.none())` generating the inputs. After that I added more tests of my own:

- **@example(...)** lines for tricky inputs I wanted to be tested every time: None, empty string, 'abc', '3-', '1-2-3', '-3-2', Arabic-Indic digits, a superscript digit and a 5000-digit number.
- **Any-type tests** that feed None, ints, floats (including NaN), bools, bytes, lists and dicts to each function.
- **Property tests** for valid input: `parse_score` round trip, `game_lead` is antisymmetric and always between -3 and 3, `set_spread` equals max-min and is never negative.
- **Rejection tests**: invalid input should give `None`, not an exception.
- **Feature tests** for what I added in the fix: advantage scoring (AD), the `strict=True` mode, the size limits, and the command line `main()` (re-prompting, end of input, Ctrl+C, running as a script).

In total the file has 28 tests. The complete test file is in Appendix B.

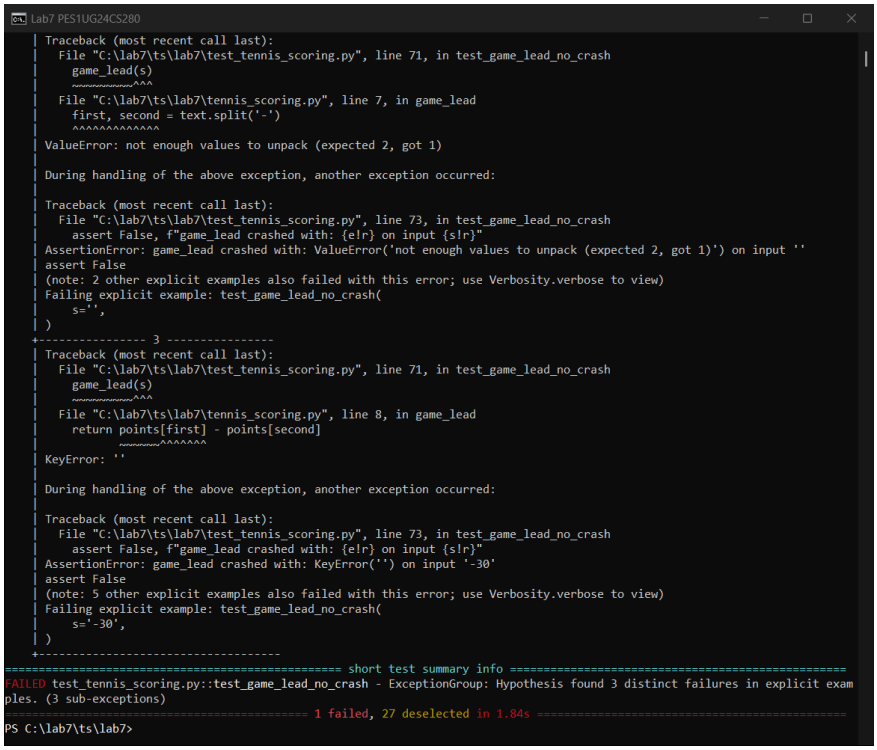
5. Step 4 - Running the tests on the buggy code

Command: `python -m pytest test_tennis_scoring.py -v --tb=no`. Result: **18 failed, 10 passed**. The tests that pass are the ones for normal valid input, which confirms the original code works for normal use. The failures are the bad-input tests (the three required fuzz tests and the extra ones) plus the tests for features the original never had: AD scoring, strict mode and a robust command line. (Note: `-rN` only hides pytest's extra summary list so the whole run fits in one screenshot.)



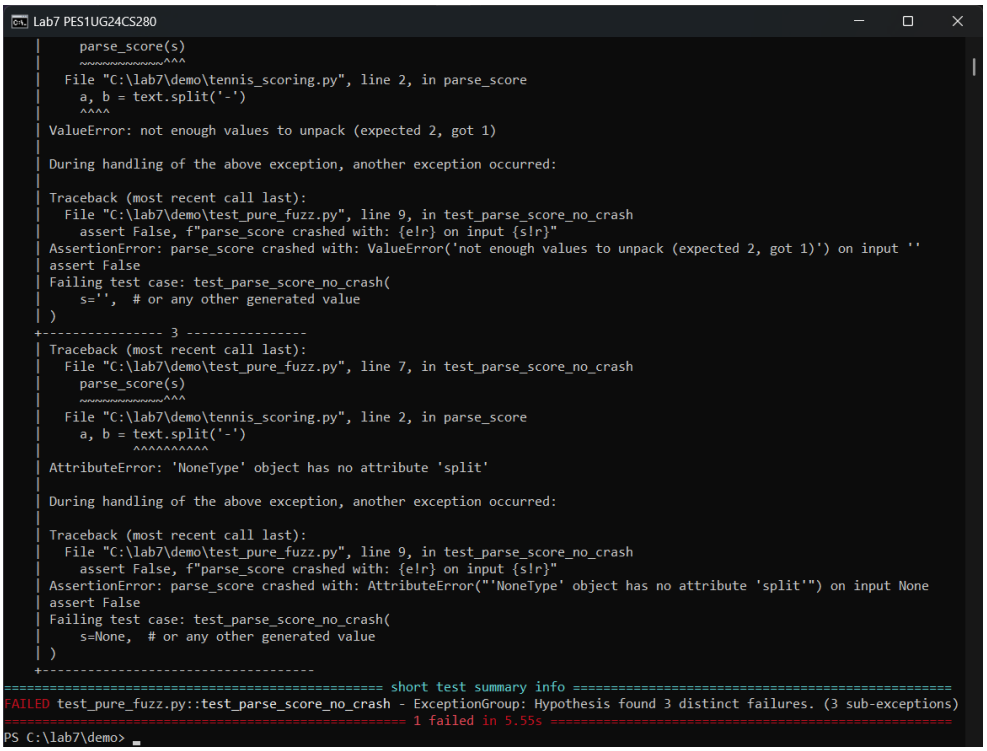
Screenshot 4: pytest on the original code - 18 failed, 10 passed

Here is the plain failure output for one test (`test_game_lead_no_crash`), showing the exact inputs that broke `game_lead`:



Screenshot 5: failure details for game_lead - ValueError on '' and KeyError on '-30'

Because my file contains `@example` lines, some of those failures come from my own examples. To check that Hypothesis finds bugs by itself, I made a tiny separate file with only the basic `@given(st.text() | st.none())` test for `parse_score` and no examples. Hypothesis found and reported failing inputs ('' and None) on its own:



Screenshot 6: Hypothesis finding parse_score crashes without any hand-written examples

6. Bugs found

I verified each of these by calling the original functions directly with the input shown. The error messages are what Python actually printed.

Function	Input	What happens in the original code
parse_score	None	AttributeError: 'NoneType' object has no attribute 'split'
parse_score	'' or 'abc'	ValueError: not enough values to unpack (expected 2, got 1)
parse_score	'1-2-3' or '-3-2'	ValueError: too many values to unpack (expected 2)
parse_score	'3-', '-', 'a-b', '1.5-2'	ValueError: invalid literal for int() with base 10
parse_score	Arabic-Indic digits, e.g. U+0663-U+0662	No error, but accepted as (3, 2) - non-ASCII digits get through
parse_score	superscript two, e.g. U+00B2-1	ValueError: invalid literal for int()
parse_score	'9'*5000 + '-1'	ValueError: Exceeds the limit (4300 digits) for integer string conversion
parse_score	b'3-2' (bytes)	TypeError: a bytes-like object is required, not 'str'
game_lead	None	AttributeError: 'NoneType' object has no attribute 'split'
game_lead	'' or '40'	ValueError: not enough values to unpack
game_lead	'40-30-15'	ValueError: too many values to unpack
game_lead	'50-30', 'AD-40', ' 40 - 30', '-30'	KeyError (label not in the points dictionary)
game_lead	b'40-30'	TypeError: a bytes-like object is required, not 'str'
set_spread	None	AttributeError: 'NoneType' object has no attribute 'split'
set_spread	'', ',', '6,,4', '6,4,'	ValueError: invalid literal for int() with base 10: ''
set_spread	'six,four', '1.5,2', superscript two	ValueError: invalid literal for int()
set_spread	'6,-4'	No error, but returns 10 - a negative number of games is accepted silently
set_spread	'9'*5000 + ',1'	ValueError: Exceeds the limit (4300 digits)
set_spread	b'6,4' (bytes)	TypeError: a bytes-like object is required, not 'str'

Root causes: the code assumes the input is always a string in exactly the right format. It never checks the type, it splits on '-' or ',' and unpacks blindly, calls `int()` on whatever comes out, looks up point labels in a dictionary without checking, and never limits the size of the numbers (very long digit strings are a denial-of-service risk).

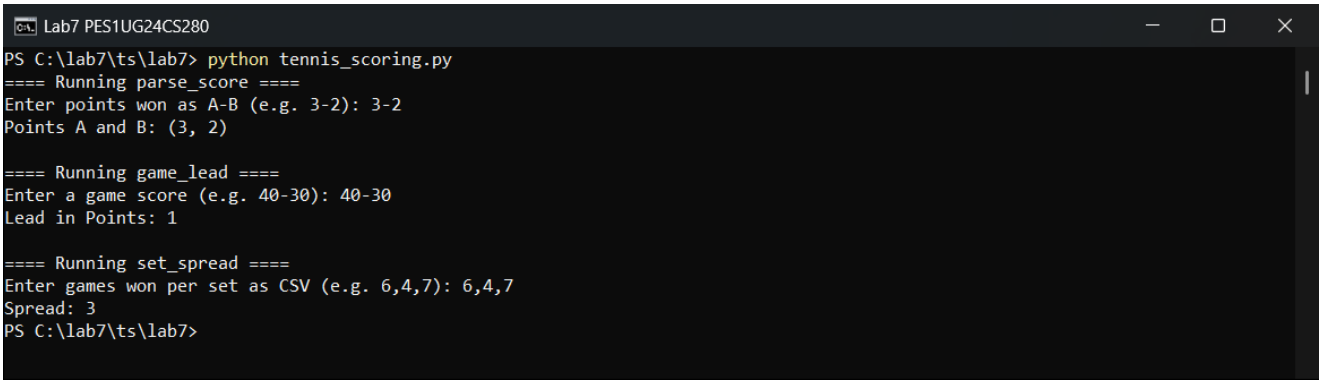
7. Step 5 - Fixing the program

I rewrote `tennis_scoring.py` with defensive programming. What I changed:

• **Type and size guard:** every function first checks that the input is a `str` of at most 100,000 characters; anything else is rejected.
• **Strict format check:** `parse_score` and `set_spread` validate the whole input with a regular expression that only allows ASCII digits 0-9 (1 to 18 digits per number) separated by '-' or ','. This removes empty items, letters, decimals, negatives, Unicode digits and giant numbers in one place. 18 digits is far more than any real score but far below Python's 4300-digit `int()` limit.
• **game_lead:** splits into exactly two parts, accepts 0, 15, 30, 40 and now also **AD** (advantage, case-insensitive), so `game_lead('AD-40')` is 1 and `game_lead('40-AD')` is -1. AD is only accepted against 40, because in real tennis advantage cannot exist at any other score. No more KeyError.
• **Spaces** around values and commas are ignored (' 40 - 30 ' and '6, 4 ,7' work).
• **Two ways to handle bad input:** by default invalid input returns `None` (never raises, which is what the handout's no-crash tests need). If the caller wants to know why, `strict=True` raises a `ValueError` with a clear message instead.
• **main()** no longer crashes: if the user types something invalid it prints a message and asks again, and it exits cleanly on Ctrl+C or end of input.

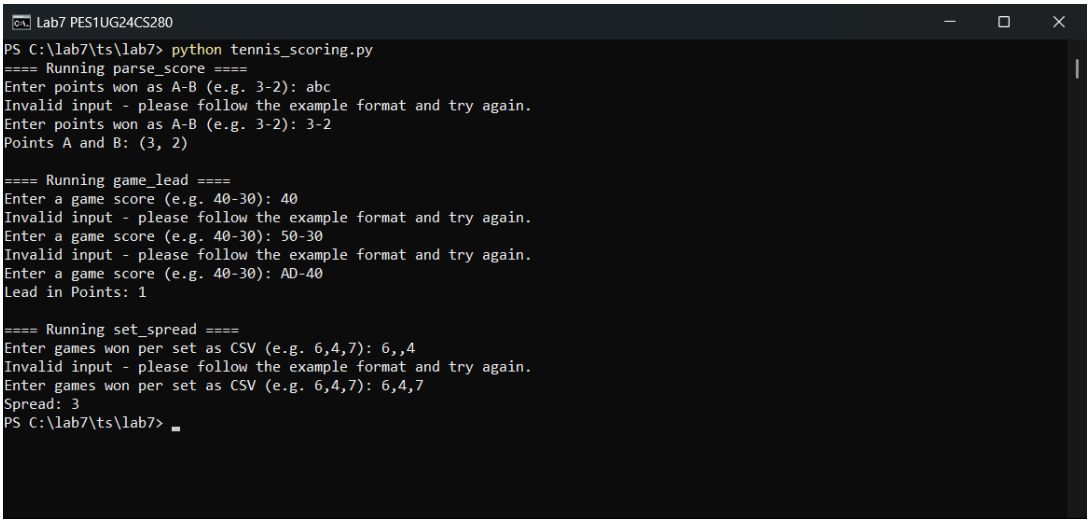
• Valid input gives exactly the same results as before.

Running the fixed program with the same valid input as before:



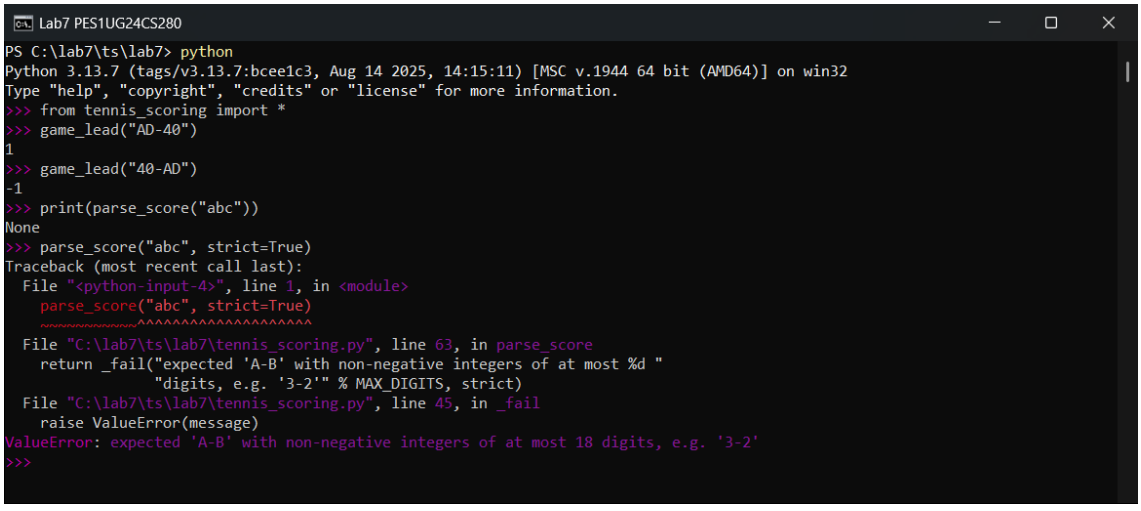
Screenshot 7: fixed program with valid input - same output as the original

And with bad inputs that used to crash it ('abc', '40', '50-30', '6,,4'). The program now explains the problem and asks again; it also accepts the new advantage score AD-40:



Screenshot 8: fixed program re-prompting on invalid input, no traceback

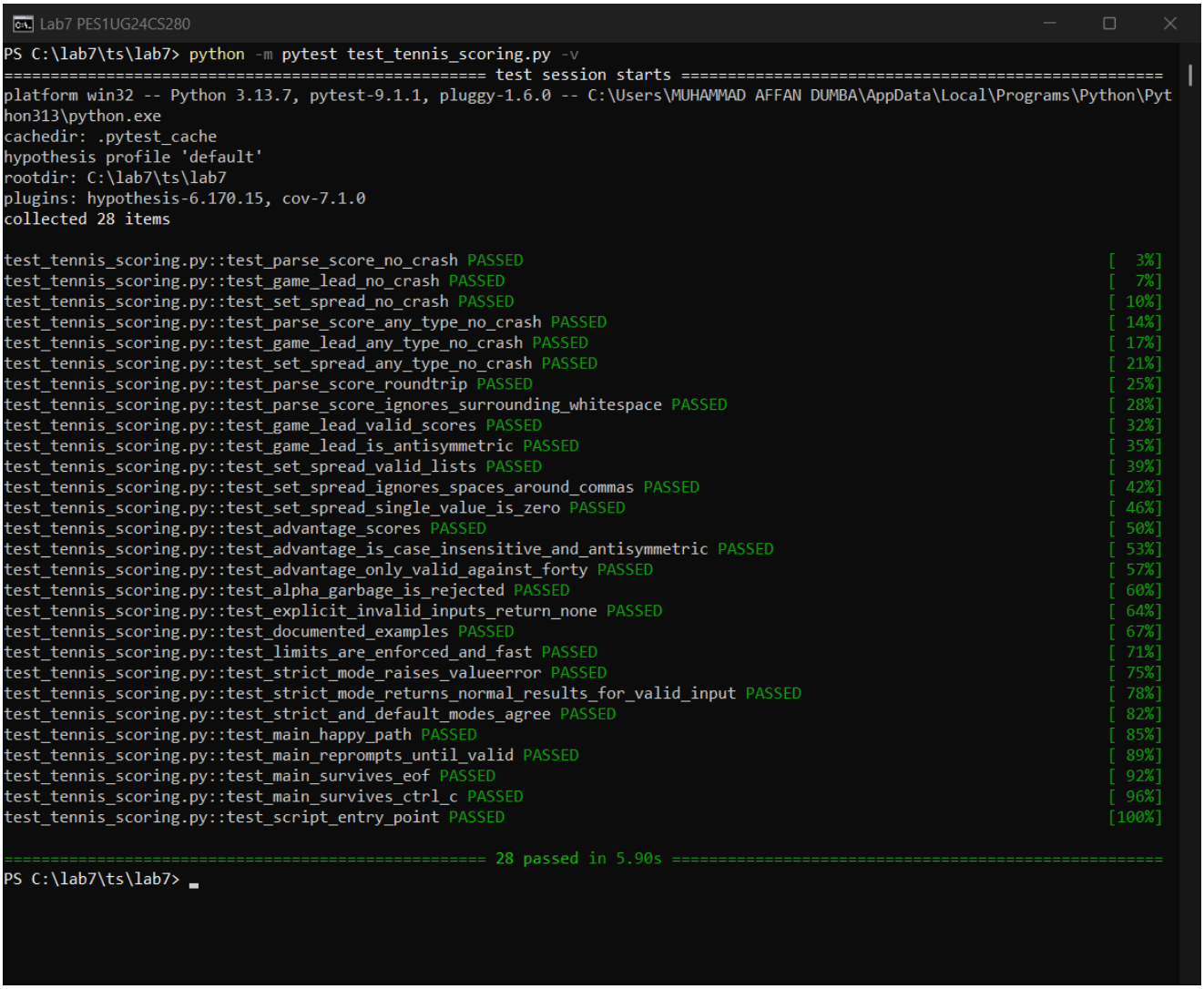
The two error-handling modes, tried in the Python shell: AD scoring works, the normal call returns None for invalid input, and strict mode raises a descriptive error:



Screenshot 9: AD scoring, None for invalid input, and ValueError in strict mode

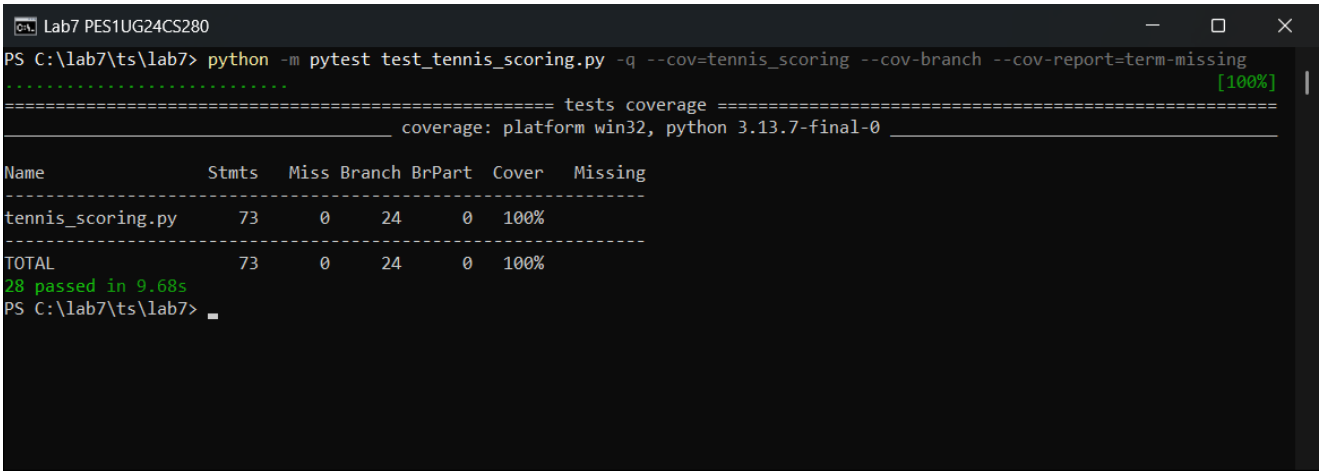
8. Step 6 - Re-running the tests on the fixed code

Command: `python -m pytest test_tennis_scoring.py -v`. Result: **28 passed**.



Screenshot 10: pytest on the fixed code - 28 passed

I also measured how much of the code the tests actually run, using `pytest-cov` (installed with `pip install pytest-cov`). Every line and every branch of `tennis_scoring.py` is covered:



Screenshot 11: coverage - 73 statements, 24 branches, 100%

9. Extra checks I did

Besides the screenshots above I did these extra checks (run on my own laptop; not shown as screenshots):

• Re-ran the 28 tests with several different Hypothesis random seeds - always 28 passed.
• Ran a separate script with 30,000 random inputs per check (mixed types, digits from other alphabets, odd separators) comparing all three functions against an independent regular-expression reference version, and checking that `strict=True` raises exactly when the normal call returns None. No differences.
• **Mutation check:** I deliberately broke the fixed code in 19 small ways (for example swapped the subtraction, removed the AD rule, changed the size limits, removed the Ctrl+C handler) and ran the tests each time. All 19 broken versions made the tests fail, so the tests really do guard the behaviour.
• Checked that both files parse as Python 3.8 syntax (the handout asks for 3.8+); the tests themselves were run on Python 3.13.7.

Design choices: the accepted formats are intentionally strict (ASCII digits only, up to 18 digits, AD only against 40), and the default mode returns `None` instead of raising because the handout's no-crash tests treat any exception as a failure; `strict=True` is there when an exception is wanted.

10. What I learned

Normal unit tests only check the cases I think of, and I would never have written tests for superscript digits or 5000-digit numbers. Hypothesis generated such inputs automatically, found real crashes in all three functions, and shrank them to tiny examples like '' and None, which made the bugs easy to understand. I also learned that fixing the 'obvious' crash is not enough - the silent bugs (negative games accepted, non-ASCII digits accepted) are only found when you also test properties of the output. The main takeaway is to validate input at the boundary: check the type, check the format strictly, limit the size, and reject everything else.

Deliverables: (1) this PDF, (2) fixed `tennis_scoring.py` (Appendix A), (3) updated `test_tennis_scoring.py` (Appendix B).

Appendix A - Fixed tennis_scoring.py

```
"""Tennis scoring helpers - hardened against malformed input (Lab 7).

Name : Muhammad Affan Dumba
SRN  : PES1UG24CS280

Contract
--------
Every public function accepts a ``strict`` keyword (default False):

* strict=False (default): the function is *total* - it never raises, whatever it
  is given (None, wrong type, empty text, huge text, unicode digits, ...).
    - valid input   -> the documented result
    - invalid input -> None
* strict=True: invalid input raises ``ValueError`` with a descriptive message
  instead of returning None (useful when the caller wants to know *why*).

Accepted formats
----------------
parse_score  : "A-B"          e.g. "3-2"      non-negative integers, up to 18 digits
game_lead    : "X-Y"          e.g. "40-30"    X, Y in 0, 15, 30, 40, AD (case-insensitive)
                              AD is only valid against 40 ("AD-40" or "40-AD")
set_spread   : "g1,g2,..."    e.g. "6,4,7"    non-negative integers, up to 18 digits each
Spaces around values/separators are ignored.
"""
import re

# Only plain ASCII digits are valid.  str.isdigit()/int() accept things such as
# superscripts or Arabic-Indic digits (and isdigit() even lies about some of
# them), so a strict regex is used instead.
MAX_DIGITS = 18            # far above any real score, far below Python's 4300-digit int() limit
MAX_INPUT_LEN = 100_000    # reject absurdly long input before doing any work

_NUMBER = r"[0-9]{1,%d}" % MAX_DIGITS
_SCORE_RE = re.compile(r"(%s)-(%s)" % (_NUMBER, _NUMBER))
_CSV_RE = re.compile(r"%s(?:,%s)*" % (_NUMBER, _NUMBER))
_SPACE_AROUND_COMMA_RE = re.compile(r"[ \t\r\n\f\v]*,[ \t\r\n\f\v]*")

# Point labels.  AD (advantage) counts as one point above 40.
_POINTS = {'0': 0, '15': 1, '30': 2, '40': 3, 'AD': 4}


def _fail(message, strict):
    """Reject invalid input: raise in strict mode, otherwise return None."""
    if strict:
        raise ValueError(message)
    return None


def _clean(text):
    """Return stripped text if `text` is a reasonably sized str, else None."""
    if not isinstance(text, str) or len(text) > MAX_INPUT_LEN:
        return None
    return text.strip()


def parse_score(text, strict=False):
    """Parse 'A-B' (e.g. '3-2') into (3, 2).  Returns None if invalid."""
    cleaned = _clean(text)
    if cleaned is None:
        return _fail("expected a string of at most %d characters" % MAX_INPUT_LEN, strict)
    m = _SCORE_RE.fullmatch(cleaned)
    if m is None:
        return _fail("expected 'A-B' with non-negative integers of at most %d "
                     "digits, e.g. '3-2'" % MAX_DIGITS, strict)
    return int(m.group(1)), int(m.group(2))


def _label_value(label):
    """Numeric value of a point label ('0','15','30','40','AD'), else None."""
    # .upper() is safe here: no non-ASCII character upper-cases to "AD".
    return _POINTS.get(label.strip().upper())


def game_lead(text, strict=False):
    """Lead of the first player in a game score such as '40-30' (-> 1).

    Valid labels: 0, 15, 30, 40 and AD (advantage, only against 40).
    Returns None if invalid.
    """
    cleaned = _clean(text)
    if cleaned is None:
        return _fail("expected a string of at most %d characters" % MAX_INPUT_LEN, strict)
    parts = cleaned.split('-')
    if len(parts) != 2:
        return _fail("expected two point labels separated by '-', e.g. '40-30'", strict)
    first, second = _label_value(parts[0]), _label_value(parts[1])
    if first is None or second is None:
        return _fail("point labels must be 0, 15, 30, 40 or AD", strict)
    # Advantage only exists against 40 (otherwise the game would be over).
    if (first == 4 and second != 3) or (second == 4 and first != 3):
        return _fail("AD is only valid against 40 ('AD-40' or '40-AD')", strict)
    return first - second


def set_spread(games_csv, strict=False):
    """Max minus min of comma separated games won, e.g. '6,4,7' -> 3.

    Returns None if the input is empty, has empty/non-numeric/negative items.
```

```
    """
    cleaned = _clean(games_csv)
    if cleaned is None:
        return _fail("expected a string of at most %d characters" % MAX_INPUT_LEN, strict)
    # tolerate spaces around the commas: '6, 4 ,7'
    cleaned = _SPACE_AROUND_COMMA_RE.sub(",", cleaned)
    if _CSV_RE.fullmatch(cleaned) is None:
        return _fail("expected comma separated non-negative integers of at most "
                     "%d digits, e.g. '6,4,7'" % MAX_DIGITS, strict)
    games = [int(g) for g in cleaned.split(',')]
    return max(games) - min(games)


# ---------------------------------------------------------------------------
# Command line interface
# ---------------------------------------------------------------------------
INVALID_MSG = "Invalid input - please follow the example format and try again."


def _ask(prompt, func, label):
    """Prompt until `func` accepts the answer.  Returns None on end of input."""
    while True:
        try:
            raw = input(prompt)
        except EOFError:          # stdin closed / piped input exhausted
            print()
            return None
        result = func(raw)
        if result is not None:
            print(label, result)
            return result
        print(INVALID_MSG)


def main():
    try:
        print("==== Running parse_score ====")
        _ask("Enter points won as A-B (e.g. 3-2): ", parse_score, "Points A and B:")

        print("\n==== Running game_lead ====")
        _ask("Enter a game score (e.g. 40-30): ", game_lead, "Lead in Points:")

        print("\n==== Running set_spread ====")
        _ask("Enter games won per set as CSV (e.g. 6,4,7): ", set_spread, "Spread:")
    except KeyboardInterrupt:
        print("\nCancelled.")


if __name__ == "__main__":
    main()
```

Appendix B - Updated test_tennis_scoring.py

```
"""Lab 7 - Fuzz tests for tennis_scoring.py using Hypothesis.

Name : Muhammad Affan Dumba
SRN  : PES1UG24CS280

Run with:  pytest test_tennis_scoring.py   (or  python -m pytest test_tennis_scoring.py)

Contract that the hardened module must honour
---------------------------------------------
* parse_score / game_lead / set_spread must NEVER raise on any input
  (default, non-strict mode).
* Valid input   -> the documented result.
* Invalid input -> None (a safe, explicit "rejected" value).
* strict=True   -> invalid input raises ValueError instead of returning None.
* main() keeps asking until it gets valid input and exits cleanly on EOF/Ctrl+C.
"""
import pytest
from hypothesis import given, example, strategies as st

import tennis_scoring
from tennis_scoring import parse_score, game_lead, set_spread

# Any kind of junk a caller might hand us (not only strings).
ANYTHING = st.one_of(
    st.none(), st.text(), st.integers(), st.floats(allow_nan=True),
    st.booleans(), st.binary(),
    st.lists(st.integers(), max_size=5),
    st.dictionaries(st.text(max_size=3), st.integers(), max_size=3),
)

POINT_LABELS = ["0", "15", "30", "40"]


# ---------------------------------------------------------------------------
# 1. Required "no crash" fuzz tests (one per function) - text | None
#    @example(...) pins counter-examples found against the original buggy code,
#    so they are always replayed as regression tests.
# ---------------------------------------------------------------------------
@given(st.text() | st.none())
@example(None)
@example("")
@example("abc")
@example("3-")
@example("-")
@example("1-2-3")
@example("a-b")
@example("-3-2")
@example("\u0663-\u0662")        # Arabic-Indic digits
@example("\u00b2-1")             # superscript two: str.isdigit() True, int() fails
@example("9" * 5000 + "-1")      # int() digit-limit DoS (Python >= 3.11)
def test_parse_score_no_crash(s):
    try:
        parse_score(s)
    except Exception as e:
        assert False, f"parse_score crashed with: {e!r} on input {s!r}"


@given(st.text() | st.none())
@example(None)
@example("")
@example("40")
@example("40-")
@example("-30")
@example("40-30-15")
@example("50-30")
@example("AD-40")
@example(" 40 - 30 ")
@example("\x00-\x00")
def test_game_lead_no_crash(s):
    try:
        game_lead(s)
    except Exception as e:
        assert False, f"game_lead crashed with: {e!r} on input {s!r}"


@given(st.text() | st.none())
@example(None)
@example("")
@example(",")
@example("6,,4")
@example("6,4,")
@example("six,four")
@example("6,-4")
@example("1.5,2")
@example("\u00b2,1")
@example("9" * 5000 + ",1")
def test_set_spread_no_crash(s):
    try:
        set_spread(s)
    except Exception as e:
        assert False, f"set_spread crashed with: {e!r} on input {s!r}"


# ---------------------------------------------------------------------------
# 2. Wider fuzzing: arbitrary (non-string) types must not crash either.
# ---------------------------------------------------------------------------
@given(ANYTHING)
def test_parse_score_any_type_no_crash(x):
```

```
    parse_score(x)


@given(ANYTHING)
def test_game_lead_any_type_no_crash(x):
    game_lead(x)


@given(ANYTHING)
def test_set_spread_any_type_no_crash(x):
    set_spread(x)


# ---------------------------------------------------------------------------
# 3. Property tests: valid input must give correct answers.
# ---------------------------------------------------------------------------
@given(st.integers(0, 10 ** 18 - 1), st.integers(0, 10 ** 18 - 1))
def test_parse_score_roundtrip(a, b):
    assert parse_score(f"{a}-{b}") == (a, b)


@given(st.integers(0, 99), st.integers(0, 99),
       st.text(" \t", max_size=3), st.text(" \t", max_size=3))
def test_parse_score_ignores_surrounding_whitespace(a, b, pad1, pad2):
    assert parse_score(f"{pad1}{a}-{b}{pad2}") == (a, b)


@given(st.sampled_from(POINT_LABELS), st.sampled_from(POINT_LABELS))
def test_game_lead_valid_scores(x, y):
    expected = POINT_LABELS.index(x) - POINT_LABELS.index(y)
    assert game_lead(f"{x}-{y}") == expected
    assert -3 <= game_lead(f"{x}-{y}") <= 3


@given(st.sampled_from(POINT_LABELS), st.sampled_from(POINT_LABELS))
def test_game_lead_is_antisymmetric(x, y):
    assert game_lead(f"{x}-{y}") == -game_lead(f"{y}-{x}")


@given(st.lists(st.integers(0, 10 ** 18 - 1), min_size=1, max_size=20))
def test_set_spread_valid_lists(games):
    csv = ",".join(map(str, games))
    result = set_spread(csv)
    assert result == max(games) - min(games)
    assert result >= 0


@given(st.lists(st.integers(0, 99), min_size=1, max_size=10),
       st.text(" \t", max_size=2))
def test_set_spread_ignores_spaces_around_commas(games, pad):
    csv = (pad + "," + pad).join(map(str, games))
    assert set_spread(csv) == max(games) - min(games)


@given(st.integers(0, 99))
def test_set_spread_single_value_is_zero(g):
    assert set_spread(str(g)) == 0


# ---------------------------------------------------------------------------
# 4. Advantage (AD) scoring
# ---------------------------------------------------------------------------
def test_advantage_scores():
    assert game_lead("AD-40") == 1
    assert game_lead("40-AD") == -1
    assert game_lead("ad-40") == 1
    assert game_lead(" Ad - 40 ") == 1


@given(st.sampled_from(["AD", "ad", "Ad", "aD"]))
def test_advantage_is_case_insensitive_and_antisymmetric(ad):
    assert game_lead(f"{ad}-40") == 1
    assert game_lead(f"40-{ad}") == -1


def test_advantage_only_valid_against_forty():
    for bad in ("AD-AD", "AD-30", "AD-15", "AD-0", "0-AD", "15-AD", "30-AD"):
        assert game_lead(bad) is None, bad


# ---------------------------------------------------------------------------
# 5. Invalid input must be rejected (None) rather than guessed at.
# ---------------------------------------------------------------------------
@given(st.text(alphabet="abcxyz,-+ .", max_size=12))
def test_alpha_garbage_is_rejected(s):
    assert parse_score(s) is None
    assert game_lead(s) is None
    assert set_spread(s) is None


BAD_SCORES = (None, "", "abc", "3-", "-3", "1-2-3", "-3-2", "3--2", "1.5-2",
              "\u00b2-1", "\u0663-\u0662", "9" * 5000 + "-1", "9" * 19 + "-1",
              "1-" + "9" * 19, b"3-2", 5, ["3-2"], "3 2", "+3-2")
BAD_LEADS = (None, "", "40", "40-", "-30", "40-30-15", "50-30", "15--0", "4-0",
             "00-30", "\u0663-30", "A-40", "AD-AD", "AD-30", b"40-30", 40, "40 30")
BAD_SPREADS = (None, "", ",", "6,,4", "6,4,", ",6", "six,four", "6,-4", "1.5,2",
               "\u00b2,1", "9" * 5000 + ",1", "9" * 19 + ",1", b"6,4", 6, [6, 4],
               "6 4", "6;4", "+6,4")


def test_explicit_invalid_inputs_return_none():
```

```
    for bad in BAD_SCORES:
        assert parse_score(bad) is None, bad
    for bad in BAD_LEADS:
        assert game_lead(bad) is None, bad
    for bad in BAD_SPREADS:
        assert set_spread(bad) is None, bad


def test_documented_examples():
    assert parse_score("3-2") == (3, 2)
    assert game_lead("40-30") == 1
    assert game_lead("0-40") == -3
    assert set_spread("6,4,7") == 3


def test_limits_are_enforced_and_fast():
    assert parse_score("9" * 18 + "-0") == (10 ** 18 - 1, 0)       # largest accepted
    assert parse_score("9" * 19 + "-0") is None                    # one digit too many
    assert set_spread(",".join(["1"] * 40_000)) == 0                # long but valid, linear time
    assert set_spread("1," * 60_000) is None                       # too long (> 100,000 chars)
    assert parse_score(" " * 200_000 + "1-2") is None
    assert parse_score("1-2" + " " * 1000) == (1, 2)


# ------------------------------------------------------------------------
# 6. strict=True: invalid input raises ValueError (with a message).
# ------------------------------------------------------------------------
def test_strict_mode_raises_valueerror():
    for fn, bad in ((parse_score, "abc"), (parse_score, None), (game_lead, "50-30"),
                    (game_lead, "AD-30"), (game_lead, "40"), (game_lead, 7),
                    (set_spread, "6,,4"), (set_spread, ""), (set_spread, b"1")):
        with pytest.raises(ValueError) as info:
            fn(bad, strict=True)
        assert str(info.value)


def test_strict_mode_returns_normal_results_for_valid_input():
    assert parse_score("3-2", strict=True) == (3, 2)
    assert game_lead("40-AD", strict=True) == -1
    assert set_spread("6, 4 ,7", strict=True) == 3


@given(ANYTHING)
def test_strict_and_default_modes_agree(x):
    for fn in (parse_score, game_lead, set_spread):
        expected = fn(x)
        if expected is None:
            with pytest.raises(ValueError):
                fn(x, strict=True)
        else:
            assert fn(x, strict=True) == expected


# ------------------------------------------------------------------------
# 7. Command line interface (main) - re-prompts, handles EOF and Ctrl+C.
# ------------------------------------------------------------------------
def _feed(monkeypatch, answers):
    it = iter(answers)

    def fake_input(prompt=""):
        try:
            item = next(it)
        except StopIteration:
            raise EOFError
        if isinstance(item, BaseException):
            raise item
        return item
    monkeypatch.setattr("builtins.input", fake_input)


def test_main_happy_path(monkeypatch, capsys):
    _feed(monkeypatch, ["3-2", "40-30", "6,4,7"])
    tennis_scoring.main()
    out = capsys.readouterr().out
    assert "(3, 2)" in out and "Lead in Points: 1" in out and "Spread: 3" in out
    assert "Invalid" not in out


def test_main_reprompts_until_valid(monkeypatch, capsys):
    _feed(monkeypatch, ["abc", "", "3-2", "40", "50-30", "AD-40", "6,,4", "6,4,7"])
    tennis_scoring.main()
    out = capsys.readouterr().out
    assert out.count("Invalid input") == 5
    assert "(3, 2)" in out and "Lead in Points: 1" in out and "Spread: 3" in out
    assert "Traceback" not in out


def test_main_survives_eof(monkeypatch, capsys):
    _feed(monkeypatch, ["bad"])           # then input() raises EOFError
    tennis_scoring.main()                  # must not raise
    assert "Invalid input" in capsys.readouterr().out


def test_main_survives_ctrl_c(monkeypatch, capsys):
    _feed(monkeypatch, ["3-2", KeyboardInterrupt()])
    try:
        tennis_scoring.main()
    except KeyboardInterrupt:              # main() must handle Ctrl+C itself
        pytest.fail("main() let KeyboardInterrupt escape")
    assert "Cancelled" in capsys.readouterr().out
def test_script_entry_point(monkeypatch, capsys):
```

```
    """`python tennis_scoring.py` runs main() when executed as a script."""
    import runpy
    _feed(monkeypatch, ["3-2", "40-30", "6,4,7"])
    runpy.run_path(tennis_scoring.__file__, run_name="__main__")
    assert "Spread: 3" in capsys.readouterr().out
```